

139. Word Break Medium

Acceptance: 49.0% | [LeetCode](#)

Given a string `s` and a dictionary of strings `wordDict`, return `true` if `s` can be segmented into a space-separated sequence of one or more dictionary words.

Note that the same word in the dictionary may be reused multiple times in the segmentation.

Example 1:

```
**Input:** s = "leetcode", wordDict = ["leet","code"]
**Output:** true
**Explanation:** Return true because "leetcode" can be segmented as "leet code".
```

Example 2:

```
**Input:** s = "applepenapple", wordDict = ["apple","pen"]
**Output:** true
**Explanation:** Return true because "applepenapple"
```

can be segmented as "apple pen apple".
Note that you are allowed to reuse a dictionary word.

Example 3:

```
**Input:** s = "catsandog", wordDict = ["cats","dog","sand","and","cat"]
**Output:** false
```

Constraints:

- `1 <= s.length <= 300`
- `1 <= wordDict.length <= 1000`
- `1 <= wordDict[i].length <= 20`
- `s` and `wordDict[i]` consist of only lowercase English letters.
- All the strings of `wordDict` are **unique**.

Claude Opus Solution

Word Break - Complete Solution Guide

1. Intuition

The key insight is recognizing this as a **decomposition problem**: a string can be segmented if we can split it into a **first valid word** + a **remaining segmentable substring**.

This creates **overlapping subproblems**: - "Can `s[0:i]` be segmented?" depends on "Can `s[0:j]` be segmented?" for some earlier `j`. - The same prefix gets checked repeatedly → perfect for **Dynamic Programming**.

The core recurrence:

`s[0:i]` is breakable **IF** there exists some split point `j` where: - `s[0:j]` is breakable (already solved subproblem), **AND** - `s[j:i]` is a word in the dictionary.

The empty string (`s[0:0]`) is trivially breakable — this is our **base case**.

2. Approach (Pseudocode)

1. Convert `wordDict` to a SET for $O(1)$ lookups.
2. Compute `max_word_length` to limit inner loop range (optimization).
3. Create `dp[]` array of size `len(s)+1`:
 - `dp[0] = True` (empty string is always breakable)
 - `dp[i] = True` if `s[0:i]` can be segmented
4. For each end position `i` from 1 to `len(s)`:
 - For each start position `j` (only check feasible word lengths):
 - If `dp[j]` is True AND `s[j:i]` is in word set:
 - `dp[i] = True`
 - break (one valid split is enough)
5. Return `dp[len(s)]`

Visualization for `s = "leetcode"`:

```
dp[0] = True      (empty)
dp[4] = True      ("leet" is a word, dp[0]=True)
dp[8] = True      ("code" is a word, dp[4]=True)
Answer: dp[8] = True ✓
```

3. Python Solution

```
from typing import List

class Solution:
    def wordBreak(self, s: str, wordDict: List[str]) -> bool:
        # Convert to set for O(1) average-case substring lookups
        words = set(wordDict)

        # Optimization: we never need to check substrings longer than
        # the longest dictionary word, so we cap the inner loop range.
        max_word_length = max(len(word) for word in words)

        n = len(s)

        # dp[i] = True means s[0:i] (first i characters) can be segmented.
        # dp[0] = True is the base case: empty string is trivially breakable.
        dp = [True] + [False] * n

        # i is the END index of the prefix we're trying to validate.
        # We build solutions for longer prefixes using shorter ones.
        for i in range(1, n + 1):
            # j is the START of the LAST word in this prefix.
            # We only look back up to max_word_length characters because
            # any word longer than that can't exist in the dictionary.
            for j in range(max(0, i - max_word_length), i):
                # If s[0:j] is breakable AND s[j:i] is a dictionary word,
                # then s[0:i] is breakable too.
                if dp[j] and s[j:i] in words:
                    dp[i] = True
                    break # One valid segmentation is enough; stop early.

        # dp[n] tells us if the entire string s[0:n] is breakable.
        return dp[n]
```

4. Complexity Analysis

Time Complexity: $O(n^2 \cdot L)$ or loosely $O(n^3)$

- Outer loop runs n times (each end position i).
- Inner loop runs up to $\min(i, \text{max_word_length})$ times.
- Each iteration does a **substring slice** $s[j:i]$ costing $O(L)$ where L is word length (up to 20).
- With the `max_word_length` optimization: $O(n \cdot \text{max_word_length} \cdot L)$.
- Without it (general case): $O(n^2)$ pairs $\times O(n)$ slice/hash = $O(n^3)$.

For this problem, since `max_word_length` ≤ 20 , it's effectively $O(n \cdot 20 \cdot 20) = O(n)$ in practice — very fast!

Space Complexity: $O(n + m)$

- `dp` array: $O(n)$.
- `words` set: $O(m)$ where m is total characters in dictionary.

5. Alternative Approaches

A) Top-Down Memoization (Recursion + Cache)

Same recurrence, but recursive. Often more intuitive:

```
from functools import lru_cache

class Solution:
    def wordBreak(self, s: str, wordDict: List[str]) -> bool:
        words = set(wordDict)

        @lru_cache(maxsize=None)
        def can_break(start: int) -> bool:
            if start == len(s): # Reached the end → success
                return True
```

```

    for end in range(start + 1, len(s) + 1):
        # If first chunk is a word AND the rest is breakable
        if s[start:end] in words and can_break(end):
            return True
    return False

return can_break(0)

```

Same complexity, but uses recursion stack space $O(n)$.

B) BFS (Graph Traversal)

Treat string indices as **nodes**; an edge from i to j exists if $s[i:j]$ is a word. Use BFS to check if we can reach index n from index 0 . Avoids revisiting indices with a `visited` set.

C) Trie-Based Optimization

Build a **Trie** from `wordDict`. While extending substrings, traverse the Trie character-by-character — this avoids generating substring slices and lets you **early-terminate** when no word starts with the current prefix. Best when there are many words sharing prefixes.

🔑 Key Pattern Takeaway

This is a classic **"interval/partition DP"** pattern:

"Can the whole be built from a valid piece + a valid remainder?"

Recognizing this recurrence (and the empty-string base case) is the transferable skill — it applies to problems like **Word Break II**, **Palindrome Partitioning**, and **Concatenated Words**.

Codex Solution

1. Intuition

This is a classic **Dynamic Programming on prefixes** problem.

We want to know whether the string `s` can be split into valid dictionary words.

The key idea:

If some prefix $s[:j]$ can already be segmented, and the remaining part $s[j:i]$ is a dictionary word, then $s[:i]$ can also be segmented.

So we define:

`dp[i]` = True if $s[:i]$ can be segmented into dictionary words

For example:

```

s = "leetcode"
wordDict = ["leet", "code"]

dp[4] = True because "leet" is a word
dp[8] = True because dp[4] is True and "code" is a word

```

Finally, the answer is `dp[len(s)]`.

2. Approach

DP Definition

`dp[i]` = whether the prefix $s[0:i]$ can be broken into dictionary words

Base Case

`dp[0]` = True

This means the empty prefix is valid. It helps us build answers for longer prefixes.

Transition

For every ending index `i`, try every possible word length.

If:

```
dp[i - length] == True
```

and:

```
s[i - length : i] exists in wordDict
```

then:

```
dp[i] = True
```

Pseudocode

```
wordSet = set(wordDict)
wordLengths = unique lengths of words in wordDict

dp = array of False of size len(s) + 1
dp[0] = True

for i from 1 to len(s):
    for length in wordLengths:
        if length <= i:
            start = i - length

            if dp[start] is True and s[start:i] is in wordSet:
                dp[i] = True
                break

return dp[len(s)]
```

3. Python Solution

```
from typing import List

class Solution:
    def wordBreak(self, s: str, wordDict: List[str]) -> bool:
        # Convert dictionary list to a set for O(1) average lookup.
        word_set = set(wordDict)

        # Store only the unique word lengths.
        # This avoids checking every possible split point unnecessarily.
        word_lengths = set(len(word) for word in wordDict)

        n = len(s)

        # dp[i] means:
        # Can the prefix s[:i] be segmented into dictionary words?
        dp = [False] * (n + 1)

        # Empty string can always be segmented.
        # This is our base case.
        dp[0] = True

        # Try to determine whether each prefix s[:i] can be segmented.
        for i in range(1, n + 1):
            # Try every possible dictionary word length.
            for length in word_lengths:
                # If the word length is greater than current prefix length,
                # it cannot be used here.
                if length > i:
                    continue

                # Candidate word would be s[start:i].
                start = i - length
```

```

    # Two conditions must hold:
    # 1. s[:start] can already be segmented.
    # 2. s[start:i] is a dictionary word.
    if dp[start] and s[start:i] in word_set:
        dp[i] = True

    # No need to check more splits for this i.
    break

# The full string can be segmented if dp[n] is True.
return dp[n]

```

4. Complexity Analysis

Let:

```

n = len(s)
L = maximum length of a word in wordDict

```

Given the constraints:

```

n <= 300
L <= 20

```

Time Complexity

For each index `i`, we try possible word lengths.

There can be at most `L` unique word lengths, and slicing `s[start:i]` costs up to $O(L)$.

Therefore:

```

Time Complexity:  $O(n * L^2)$ 

```

Since `L <= 20`, this is very efficient.

If substring slicing is considered $O(1)$ conceptually, many discussions simplify this to:

```

 $O(n * L)$ 

```

A more general DP that checks every split point `j` would be:

```

 $O(n^2)$ 

```

or in Python with slicing cost:

```

 $O(n^3)$ 

```

This solution is optimized by only considering valid dictionary word lengths.

Space Complexity

The DP array has size `n + 1`.

The dictionary set stores the words.

```

Space Complexity:  $O(n + \text{total characters in wordDict})$ 

```

If only considering extra DP space:

```

 $O(n)$ 

```

5. Alternative Approaches

1. Top-Down DFS with Memoization

Try to recursively break the string from a starting index.

```

dfs(start) = whether s[start:] can be segmented

```

Memoize results for each `start` to avoid repeated work.

This is essentially the same DP idea, but written recursively.

2. BFS Over Indices

Treat each index as a node.

If `s[i:j]` is a dictionary word, then there is an edge from `i` to `j`.

The goal is to see whether index `n` is reachable from index `0`.

3. Trie-Based Search

Build a Trie from `wordDict`.

For each valid DP position `i`, walk forward in the Trie to find matching dictionary words.

This avoids repeatedly slicing strings and can be efficient when the dictionary is large.

Key Pattern

This problem uses the common pattern:

Can a prefix be built from previous valid prefixes?

That is why dynamic programming is the natural fit.